

LO17 : Indexation et Recherche d'information

TD4 : Correcteur Orthographique

Printemps 2026

Travail à réaliser

Vous devez réaliser un code Python qui permettra de corriger et d'associer un lemme valide à chacun des termes d'une requête saisie au clavier. Pour ce traitement, vous définirez manuellement un fichier d'une ou deux dizaines de mots et de lemmes judicieusement choisis pour tester vos algorithmes.

L'analyseur doit :

1. Saisir la requête au clavier ;
2. Appliquer sur l'ensemble de la phrase le même traitement de tokenisation et de lemmatisation que celui effectué lors des TDs précédents ;
3. Parcourir les termes extraits un à un et réaliser les opérations suivantes :
 - (a) Tester si le terme est une entité spécifique (un nombre, une date, ...). Si oui, le conserver tel quel et passer au terme suivant ;
 - (b) Tester si le terme existe dans le lexique (votre index). Si oui, le valider directement et passer au terme suivant ;
 - (c) S'il n'existe pas (il s'agit probablement d'une faute de frappe), générer une liste de candidats via l'algorithme de recherche par préfixe vu en cours (en définir les hyperparamètres *seuilMin*, *seuilMax* et *seuilProximité*) ;
 - (d) S'il n'y a qu'un seul candidat dans cette liste, le retourner et l'associer au mot d'origine ;
 - (e) Si la liste contient plusieurs mots candidats, utiliser l'algorithme de distance de Levenshtein pour les départager, choisir le mot ayant la plus petite distance d'édition, et retourner son lemme ;
 - (f) Si aucun mot n'est trouvé (liste vide), l'indiquer à l'utilisateur.

Lorsque vos algorithmes ont été testés et validés sur votre lexique restreint, appliquez votre programme au lexique complet des lemmes du corpus de l'ADIT.

Question de réflexion

L'approche heuristique implémentée ci-dessus (utiliser la recherche par préfixe pour réduire l'espace des candidats avant d'appliquer la distance de Levenshtein) permet d'optimiser considérablement le temps de calcul. Cependant, elle possède une vulnérabilité algorithmique majeure face à un type d'erreur de frappe particulier.

Identifiez cette vulnérabilité en donnant un exemple précis de mot mal orthographié pour lequel votre algorithme échouerait systématiquement à trouver la bonne correction. Ensuite, proposez une méthode alternative qui permettrait de résoudre ce problème de manière plus robuste.